Task1_Python_File_Automation

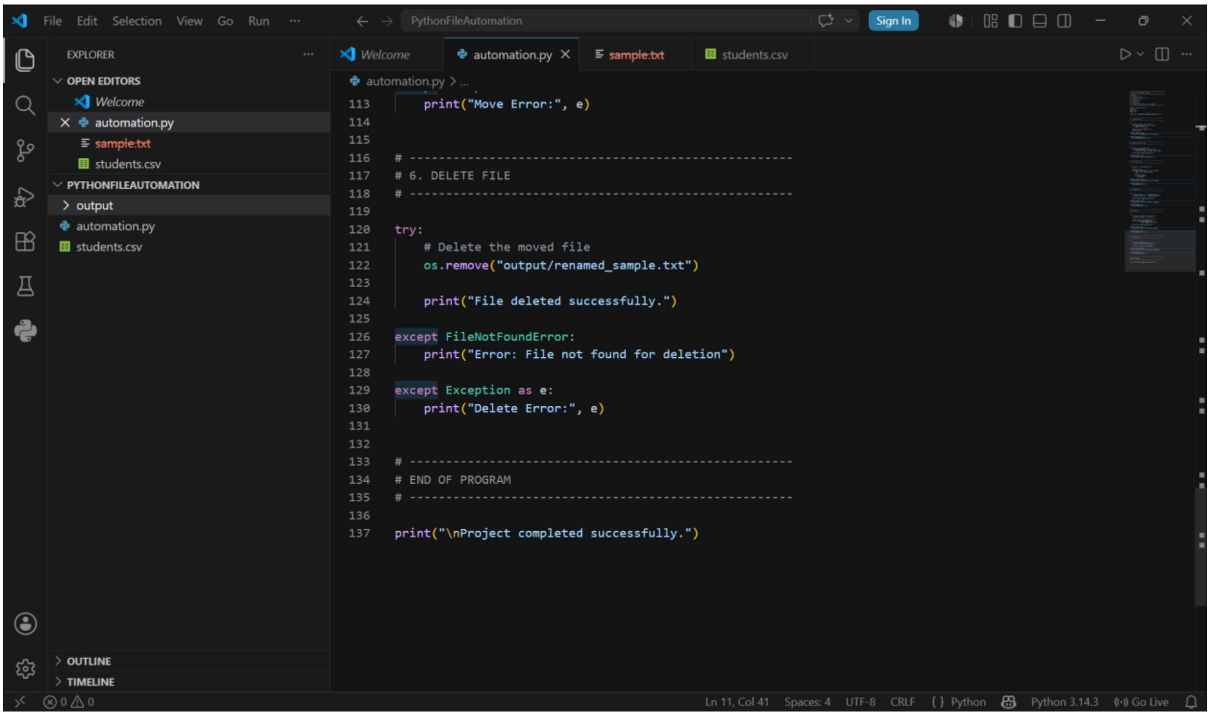
Objective

The objective of this project is to understand Python file handling, automation logic, and exception handling. This project demonstrates how to perform file operations such as reading, writing, renaming, moving, and deleting files using Python. It also helps in learning how to manage errors effectively using try-except blocks.

Technologies Used

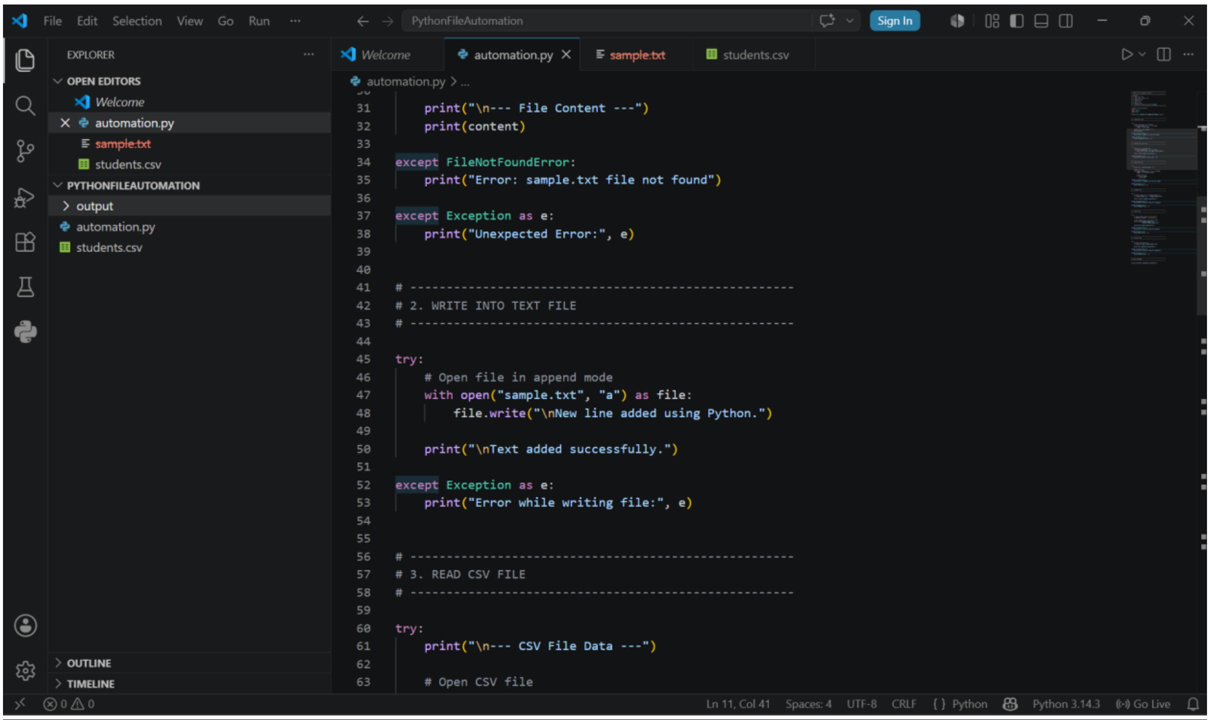
- Python
- Flask
- Pandas
- VS Code
- GitHub
- Render

Code Screenshots



```python
    print("\nFile renamed successfully.")

except FileNotFoundError:
    print("Error: File not found for renaming")

except Exception as e:
    print("Rename Error:", e)


# ---------------------------------------------------
# 5. MOVE FILE
# ---------------------------------------------------

try:
    # Create output folder if not exists
    os.makedirs("output", exist_ok=True)

    # Move renamed file into output folder
    shutil.move("renamed_sample.txt",
                "output/renamed_sample.txt")

    print("File moved successfully.")

except FileNotFoundError:
    print("Error: File not found for moving")

except Exception as e:
    print("Move Error:", e)


# ---------------------------------------------------
# 6. DELETE FILE
```

```python
# 3. READ CSV FILE
# ---------------------------------------------------

try:
    print("\n--- CSV File Data ---")

    # Open CSV file
    with open("students.csv", "r") as csvfile:
        reader = csv.reader(csvfile)

        # Read each row
        for row in reader:
            print(row)

except FileNotFoundError:
    print("Error: students.csv file not found")

except Exception as e:
    print("CSV Error:", e)


# ---------------------------------------------------
# 4. RENAME FILE
# ---------------------------------------------------

try:
    # Rename sample.txt to renamed_sample.txt
    os.rename("sample.txt", "renamed_sample.txt")

    print("\nFile renamed successfully.")

except FileNotFoundError:
    print("Error: File not found for renaming")
```


File Edit Selection View Go Run
PythonFileAutomation
Sign In
EXPLORER
OPEN EDITORS
Welcome
automation.py
sample.txt
students.csv
PYTHONFILEAUTOMATION
output
automation.py
students.csv
OUTLINE
TIMELINE
automation.py > ...
    print("\n--- File Content ---")
    print(content)

except FileNotFoundError:
    print("Error: sample.txt file not found")

except Exception as e:
    print("Unexpected Error:", e)

--
2. WRITE INTO TEXT FILE
--

try:
    # Open file in append mode
    with open("sample.txt", "a") as file:
        file.write("\nNew line added using Python.")

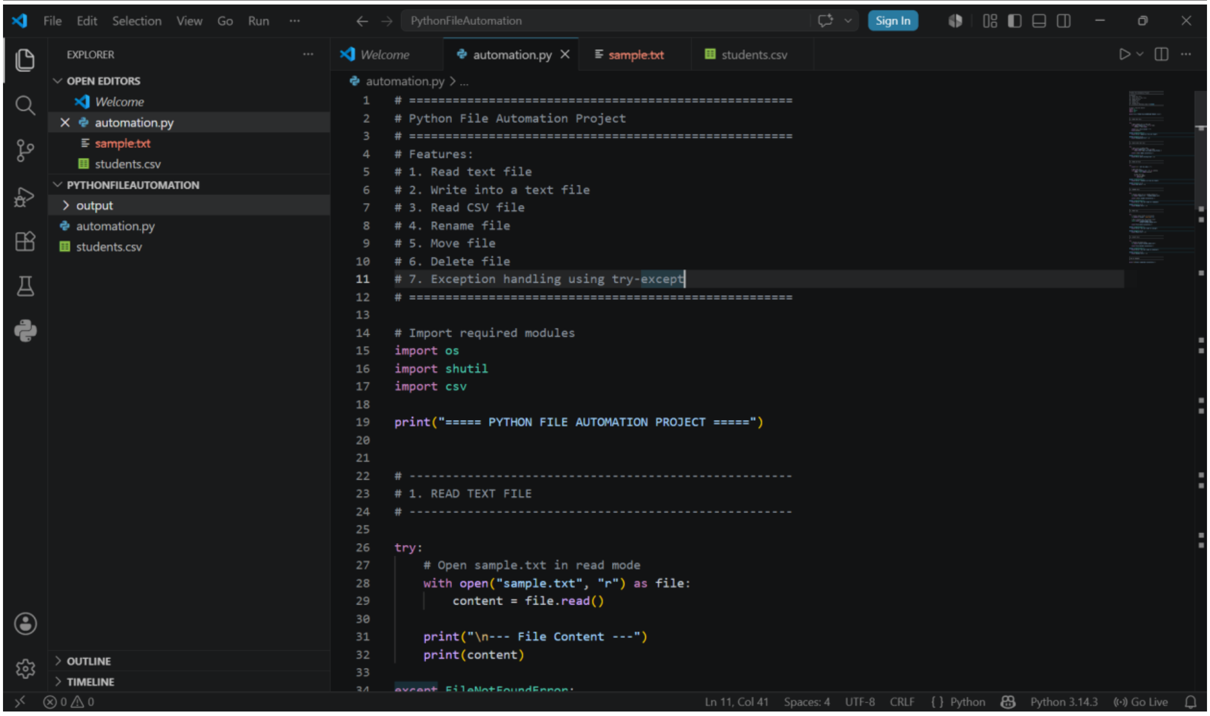
    print("\nText added successfully.")

except Exception as e:
    print("Error while writing file:", e)

--
3. READ CSV FILE
--

try:
    print("\n--- CSV File Data ---")

    # Open CSV file
Ln 11, Col 41 Spaces: 4 UTF-8 CRLF { } Python Python 3.14.3 Go Live


File Edit Selection View Go Run
PythonFileAutomation
Sign In
EXPLORER
OPEN EDITORS
Welcome
automation.py
sample.txt
students.csv
PYTHONFILEAUTOMATION
output
automation.py
students.csv
OUTLINE
TIMELINE
automation.py > ...
==
Python File Automation Project
==
Features:
1. Read text file
2. Write into a text file
3. Read CSV file
4. Rename file
5. Move file
6. Delete file
7. Exception handling using try-except
==

# Import required modules
import os
import shutil
import csv

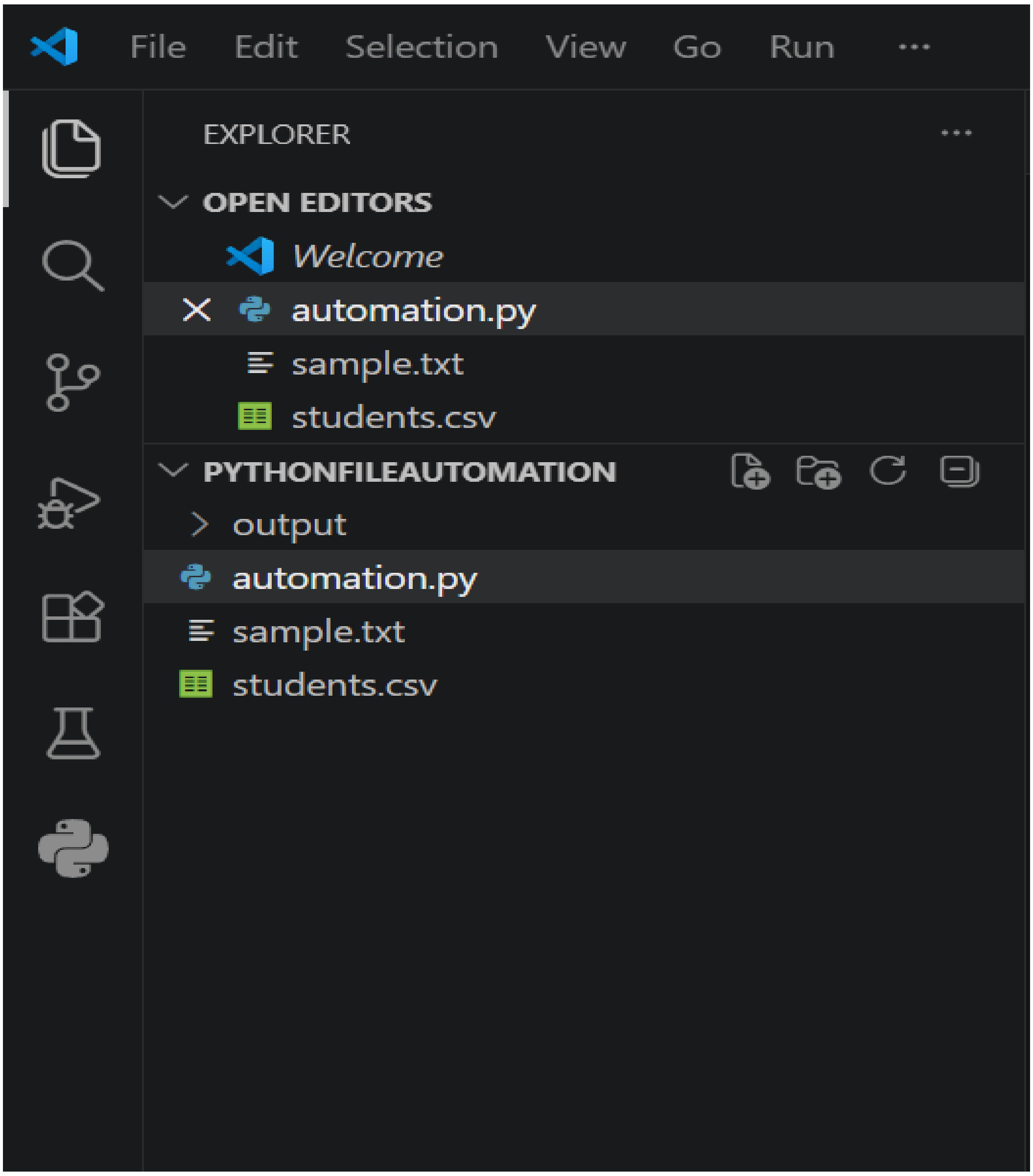
print("===== PYTHON FILE AUTOMATION PROJECT =====")

--
1. READ TEXT FILE
--

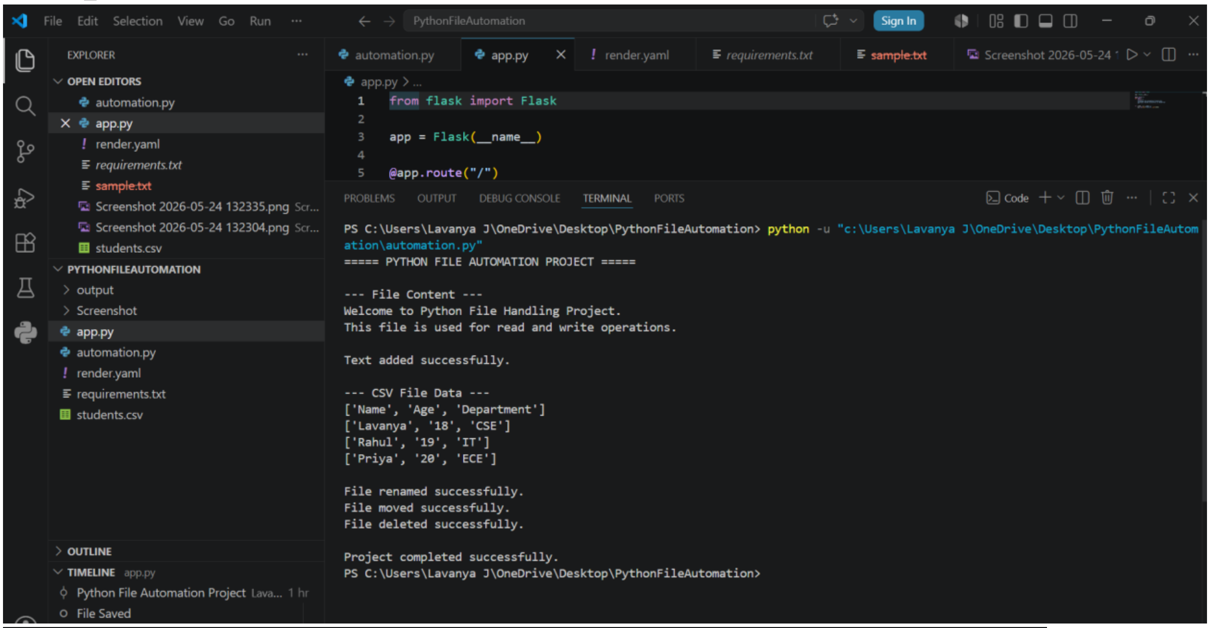
try:
    # Open sample.txt in read mode
    with open("sample.txt", "r") as file:
        content = file.read()

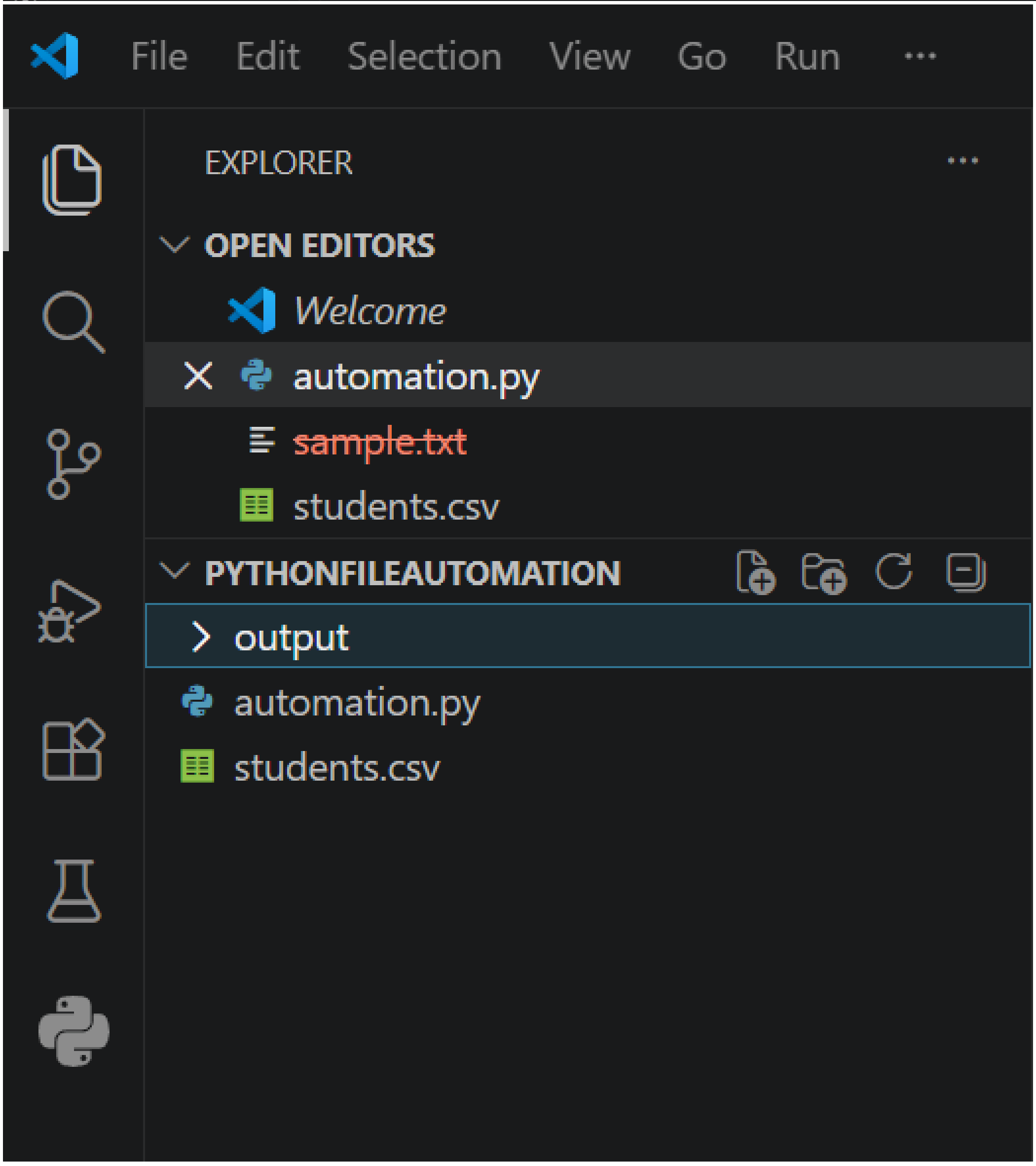
    print("\n--- File Content ---")
    print(content)

Ln 11, Col 41 Spaces: 4 UTF-8 CRLF { } Python Python 3.14.3 Go Live


File
Edit
Selection
View
Go
Run
EXPLORER
OPEN EDITORS
Welcome
automation.py
sample.txt
students.csv
PYTHONFILEAUTOMATION
output
automation.py
sample.txt
students.csv

Output Screenshots





GitHub Repository Link

https://github.com/jlavanya132008-byte/python-file-automation

Hosted Link

https://python-file-automation-1.onrender.com

Conclusion

This project helped me understand the basics of Python file handling and automation.